

PEMBELAJARAN PHP OOP MELALUI PELACAKDUIT

■ Pengantar

Dokumen ini menjelaskan konsep **Object-Oriented Programming (OOP)** dalam PHP melalui studi kasus aplikasi **PelacakDuit** — sebuah aplikasi pencatatan keuangan sederhana yang dibuat untuk tujuan pembelajaran.

Program ini dibuat dengan AI melalui kerja sama:

- **Hermes Agent** (by Nous Research)
- **9Router** dengan berbagai model AI yang tersedia

■■ **Disclaimer:** Program ini tidak memiliki fitur autentikasi/login dan hanya cocok untuk pembelajaran serta development lokal. Jangan gunakan untuk data sensitif atau production tanpa menambahkan layer autentikasi terlebih dahulu.

■ Tujuan Pembelajaran

Setelah mempelajari program ini, mahasiswa diharapkan dapat:

- Memahami konsep dasar OOP (Class, Object, Property, Method)
- Menerapkan prinsip SOLID dalam kode PHP
- Memahami pattern MVC (Model-View-Controller)
- Membuat aplikasi CRUD sederhana dengan PHP OOP
- Memahami autoloading dan namespace di PHP

■■ Struktur Program

```
pelacakduit/  
■■■ config/           # Konfigurasi database & aplikasi  
■   ■■■ config.php    # Pengaturan aplikasi  
■   ■■■ Database.php  # Kelas database (Singleton)  
■   ■■■ Schema.php    # Auto-create tables
```

```
■■■ models/                # Model (Data Layer)
■   ■■■ Account.php
■   ■■■ Category.php
■   ■■■ Transaction.php
■■■ controllers/           # Controller (Business Logic)
■   ■■■ AccountController.php
■   ■■■ CategoryController.php
■   ■■■ DashboardController.php
■   ■■■ TransactionController.php
■■■ views/                 # View (Presentation Layer)
■   ■■■ layouts/           # Header, footer, navigation
■   ■■■ accounts/          # Halaman manajemen akun
■   ■■■ categories/        # Halaman manajemen kategori
■   ■■■ dashboard/         # Dashboard utama
■   ■■■ transactions/      # Halaman transaksi
■■■ public/                # Entry point
■   ■■■ index.php          # Router sederhana
■   ■■■ .htaccess           # Apache rewrite rules
■■■ database/              # Database
    ■■■ pelacakduit.db     # SQLite database
```

■ Konsep OOP yang Dipelajari

1. Class dan Object

Apa itu Class?

Class adalah blueprint/template untuk membuat object. Class mendefinisikan property (data) dan method (fungsi) yang dimiliki oleh object.

Apa itu Object?

Object adalah instance dari class. Setiap object memiliki state (property) dan behavior (method).

Contoh di PelacakDuit:

```
// Class Database (config/Database.php)
class Database {
    private static ?PDO $instance = null;

    public static function getConnection(array $config): PDO {
        if (self::$instance === null) {
            // Buat koneksi baru
            self::$instance = new PDO(...);
        }
        return self::$instance;
    }
}

// Penggunaan
$db = Database::getConnection($config);
```

Penjelasan:

- `Database` adalah class
 - `$db` adalah object (instance dari Database)
 - Method `getConnection()` bersifat `static`, sehingga bisa dipanggil tanpa membuat object terlebih dahulu
-

2. Property dan Method

Property adalah variabel yang ada di dalam class. Property bisa memiliki visibility:

- `public` → bisa diakses dari luar class
- `private` → hanya bisa diakses dari dalam class
- `protected` → bisa diakses dari class itu sendiri dan subclass

Method adalah fungsi yang ada di dalam class.

Contoh di PelacakDuit:

```
// models/Transaction.php
class Transaction {
    private PDO $db; // Property private, hanya bisa diakses di dalam class

    public function __construct(PDO $db) { // Constructor method
        $this->db = $db;
    }

    public function getAll(int $limit = 50, int $offset = 0): array { // Public method
        $stmt = $this->db->prepare("SELECT ...");
        $stmt->execute();
        return $stmt->fetchAll();
    }
}
```

Penjelasan:

- `$db` adalah property private (enkapsulasi)
 - `__construct()` adalah constructor (method khusus yang dijalankan saat object dibuat)
 - `getAll()` adalah method public yang bisa dipanggil dari luar class
-

3. Constructor

Constructor adalah method khusus yang dijalankan otomatis saat object dibuat (`new`).

Contoh di PelacakDuit:

```
// models/Account.php
```

```

class Account {
    private PDO $db;

    public function __construct(PDO $db) {
        $this->db = $db;
    }
}

```

Penggunaan:

```
$accountModel = new Account($db); // Constructor otomatis dijalankan
```

4. Encapsulation (Enkapsulasi)

Encapsulation adalah prinsip menyembunyikan detail implementasi dan hanya menampilkan interface yang diperlukan.

Cara: Gunakan visibility `private` untuk property, dan `public` untuk method.

Contoh di PelacakDuit:

```

// config/Database.php
class Database {
    private static ?PDO $instance = null; // Private property

    public static function getConnection(array $config): PDO { // Public method
        // Implementation detail tersembunyi
        if (self::$instance === null) {
            self::$instance = new PDO(...);
        }
        return self::$instance;
    }
}

```

Keuntungan:

- Kode lebih aman (tidak bisa diubah sembarangan dari luar)
 - Mudah di-maintain (ubah internal tanpa■■■ user class)
 - Memudahkan testing (mock method)
-

5. Inheritance (Pewarisan)

Inheritance adalah mekanisme di mana class baru mewarisi property dan method dari class yang sudah ada.

Catatan: Di PelacakDuit, inheritance tidak digunakan secara aktif karena sederhana. Namun, prinsip ini penting untuk dipahami.

Contoh umum (tidak ada di program ini):

```
class Vehicle {
    protected string $brand;

    public function startEngine() {
        echo "Engine started";
    }
}

class Car extends Vehicle { // Inheritance
    public function openTrunk() {
        echo "Trunk opened";
    }
}

$car = new Car();
$car->startEngine(); // Method dari parent class
$car->openTrunk();   // Method dari child class
```

6. Interface dan Abstract Class

Interface adalah contract yang mendefinisikan method apa saja yang harus ada di class, tanpa implementasi.

Abstract Class adalah class yang tidak bisa di-instantiate dan biasanya memiliki method abstract (tanpa body).

Catatan: Di PelacakDuit, interface tidak digunakan. Namun, prinsip ini penting untuk pattern design yang lebih kompleks.

7. Autoloading dan Namespace

Apa itu Namespace?

Namespace adalah cara mengelompokkan class-class agar tidak bentrok.

Apa itu Autoloading?

Autoloading adalah mekanisme di mana PHP otomatis memuat file class saat class tersebut dipanggil.

Contoh di PelacakDuit:

```
// config/config.php
namespace PelacakDuit\Config;

class Database {
```

```

    } // ...

}

// public/index.php (autoloader)
spl_autoload_register(function ($className) {
    $classMap = [
        'Config\\' => 'config/',
        'Models\\' => 'models/',
        'Controllers\\' => 'controllers/'
    ];

    foreach ($classMap as $namespace => $dir) {
        if (strpos($className, $namespace) === 0) {
            $relativeClass = str_replace($namespace, '', $className);
            $file = BASE_PATH . '/' . $dir . strtolower(str_replace('\\', '/', $relativeClass)) . '.php';
            if (file_exists($file)) {
                require_once $file;
            }
        }
    }
});

```

Penggunaan:

```

use PelacakDuit\Models\Transaction;
use PelacakDuit\Models\Account;

$transaction = new Transaction($db);
$saccount = new Account($db);

```

8. MVC Pattern (Model-View-Controller)

PelacakDuit menggunakan pattern MVC untuk memisahkan logic dan presentation.

Model

- Bertanggung jawab atas data, logic, dan aturan bisnis
- Berinteraksi dengan database
- Contoh: `Transaction`, `Category`, `Account`

View

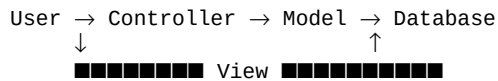
- Bertanggung jawab atas tampilan (UI)
- Hanya menampilkan data dari controller
- Contoh: `views/dashboard/index.php`, `views/transactions/index.php`

Controller

- Bertanggung jawab atas logic bisnis

- Menerima input dari user, memanggil model, dan memilih view
- Contoh: `DashboardController`, `TransactionController`

Alur MVC di PelacakDuit:



Contoh:

```

// public/index.php (router)
$page = $_GET['page'] ?? 'dashboard';

switch ($page) {
    case 'dashboard':
        $controller = new DashboardController($db);
        $data = $controller->index(); // Controller memanggil model
        require 'views/dashboard/index.php'; // View menampilkan data
        break;
}

// controllers/DashboardController.php
class DashboardController {
    private Transaction $transactionModel;

    public function __construct(PDO $db) {
        $this->transactionModel = new Transaction($db);
    }

    public function index(): array {
        $summary = $this->transactionModel->getSummary(); // Model query database
        return [
            'summary' => $summary,
            'balance' => $summary['income'] - $summary['expense']
        ];
    }
}
  
```

9. Dependency Injection

Dependency Injection adalah pattern di mana object menerima dependensinya dari luar, bukan membuatnya sendiri.

Contoh di PelacakDuit:

```

// controllers/TransactionController.php
class TransactionController {
    private Transaction $transactionModel;
    private Category $categoryModel;

    public function __construct(PDO $db) { // Inject PDO
        $this->transactionModel = new Transaction($db);
        $this->categoryModel = new Category($db);
    }
}
  
```

```
}  
}
```

Keuntungan:

- Mudah di-test (bisa mock PDO)
 - Lebih fleksibel (bisa ganti implementasi tanpa ubah class)
 - Menghindari tight coupling
-

10. Single Responsibility Principle (SRP)

SRP adalah prinsip bahwa satu class hanya boleh punya satu alasan untuk berubah (satu tanggung jawab).

Contoh di PelacakDuit:

```
// Class hanya untuk database koneksi  
class Database {  
    private static ?PDO $instance = null;  
  
    public static function getConnection(array $config): PDO {  
        // ...  
    }  
}  
  
// Class hanya untuk data transaksi  
class Transaction {  
    private PDO $db;  
  
    public function getAll(): array { /* ... */ }  
    public function create(...): int { /* ... */ }  
    public function update(...): bool { /* ... */ }  
    public function delete(...): bool { /* ... */ }  
}  
  
// Class hanya untuk data kategori  
class Category {  
    private PDO $db;  
  
    public function getAll(): array { /* ... */ }  
    public function create(...): int { /* ... */ }  
    // ...  
}
```

Bukan SRP (salah):

```
// Buruk: Class punya banyak tanggung jawab  
class Database {  
    public function getConnection() { /* ... */ }  
    public function getAllTransactions() { /* ... */ } // Ini urusan Transaction!  
    public function getAllCategories() { /* ... */ } // Ini urusan Category!  
}
```

■ Studi Kasus: Transaction Model

Mari kita telaah `Transaction.php` secara detail:

```
// models/Transaction.php
namespace PelacakDuit\Models;

use PDO;

class Transaction {
    private PDO $db;

    public function __construct(PDO $db) {
        $this->db = $db;
    }

    // GET ALL - Ambil semua transaksi dengan pagination
    public function getAll(int $limit = 50, int $offset = 0): array {
        $stmt = $this->db->prepare("
            SELECT t.*, c.name as category_name, c.color, a.name as account_name
            FROM transactions t
            JOIN categories c ON t.category_id = c.id
            JOIN accounts a ON t.account_id = a.id
            ORDER BY t.date DESC, t.id DESC
            LIMIT :limit OFFSET :offset
        ");
        $stmt->bindValue(':limit', $limit, PDO::PARAM_INT);
        $stmt->bindValue(':offset', $offset, PDO::PARAM_INT);
        $stmt->execute();
        return $stmt->fetchAll();
    }

    // GET SUMMARY - Hitung total pemasukan dan pengeluaran
    public function getSummary(): array {
        $stmt = $this->db->query("
            SELECT
                COALESCE(SUM(CASE WHEN type = 'income' THEN amount ELSE 0 END), 0) as income,
                COALESCE(SUM(CASE WHEN type = 'expense' THEN amount ELSE 0 END), 0) as expense
            FROM transactions
            WHERE date >= date('now', '-30 days')
        ");
        return $stmt->fetch();
    }

    // CREATE - Tambah transaksi baru
    public function create(
        int $categoryId,
        float $amount,
        string $type,
        ?string $description,
        string $date,
        int $accountId = 1
    ): int {
        $stmt = $this->db->prepare("
            INSERT INTO transactions (category_id, amount, type, description, date, account_id)
            VALUES (:category_id, :amount, :type, :description, :date, :accountId)
        ");
        $stmt->execute([
            ':category_id' => $categoryId,
            ':amount' => $amount,
            ':type' => $type,
            ':description' => $description,
            ':date' => $date,
            ':accountId' => $accountId
        ]);
        return (int) $this->db->lastInsertId();
    }
}
```

```

// UPDATE - Update transaksi
public function update(
    int $id,
    int $categoryId,
    float $amount,
    string $type,
    ?string $description,
    string $date,
    int $accountId
): bool {
    $stmt = $this->db->prepare("
        UPDATE transactions
        SET category_id = :category_id,
            amount = :amount,
            type = :type,
            description = :description,
            date = :date,
            account_id = :accountId
        WHERE id = :id
    ");
    return $stmt->execute([
        ':id' => $id,
        ':category_id' => $categoryId,
        ':amount' => $amount,
        ':type' => $type,
        ':description' => $description,
        ':date' => $date,
        ':accountId' => $accountId
    ]);
}

// DELETE - Hapus transaksi
public function delete(int $id): bool {
    $stmt = $this->db->prepare("DELETE FROM transactions WHERE id = :id");
    return $stmt->execute([':id' => $id]);
}
}

```

Konsep yang dipelajari:

- Constructor injection (PDO \$db)
- Prepared statements (anti-SQL injection)
- Type hints (int, float, string, ?string, array)
- Return type declaration (: array, : int, : bool)
- Encapsulation (private PDO \$db)
- Method overloading (tidak ada, tapi bisa ditambahkan)
- Single Responsibility (hanya urusan transaksi)

■ Studi Kasus: Router Sederhana

```

// public/index.php
<?php

define('BASE_PATH', __DIR__ . '/../');

// Autoloader

```

```

spl_autoload_register(function ($className) {
    // ... (lihat bagian sebelumnya)
});

// Load config
require BASE_PATH . '/config/config.php';

// Database connection
$db = Config\Database::getConnection($config['database']);

// Router
$page = $_GET['page'] ?? 'dashboard';
$action = $_GET['action'] ?? '';

switch ($page) {
    case 'dashboard':
        require BASE_PATH . '/controllers/DashboardController.php';
        $controller = new PelacakDuit\Controllers\DashboardController($db);
        $data = $controller->index();
        require BASE_PATH . '/views/dashboard/index.php';
        break;

    case 'transactions':
        require BASE_PATH . '/controllers/TransactionController.php';
        $controller = new PelacakDuit\Controllers\TransactionController($db);

        switch ($action) {
            case 'create':
                $data = $controller->create();
                require BASE_PATH . '/views/transactions/form.php';
                break;
            case 'store':
                $result = $controller->store();
                header('Content-Type: application/json');
                echo json_encode($result);
                break;
            case 'edit':
                $data = $controller->edit($id);
                require BASE_PATH . '/views/transactions/form.php';
                break;
            case 'update':
                $result = $controller->update();
                header('Content-Type: application/json');
                echo json_encode($result);
                break;
            case 'delete':
                $result = $controller->delete($id);
                header('Content-Type: application/json');
                echo json_encode($result);
                break;
            default:
                $data = $controller->index();
                require BASE_PATH . '/views/transactions/index.php';
                break;
        }
        break;

    case 'accounts':
        // ... (sama seperti transactions)
        break;

    default:
        echo "Halaman tidak ditemukan.";
        break;
}

```

Konsep yang dipelajari:

- Entry point (index.php)

- Routing sederhana dengan switch-case
 - Dynamic controller loading
 - Dependency injection ke controller
 - Separation of concerns (router, controller, view)
-

■ Latihan untuk Mahasiswa

Latihan 1: Tambah Method Baru

Soal: Tambahkan method `getById(int $id): ?array` di `Transaction` model untuk ambil satu transaksi berdasarkan ID.

Petunjuk:

```
public function getById(int $id): ?array {
    $stmt = $this->db->prepare("SELECT * FROM transactions WHERE id = :id");
    $stmt->execute([':id' => $id]);
    $result = $stmt->fetch();
    return $result ? $result : null;
}
```

Latihan 2: Tambah Filter

Soal: Tambahkan method `getByDateRange(string $startDate, string $endDate): array` di `Transaction` model.

Petunjuk:

```
public function getByDateRange(string $startDate, string $endDate): array {
    $stmt = $this->db->prepare("
        SELECT * FROM transactions
        WHERE date BETWEEN :startDate AND :endDate
        ORDER BY date DESC
    ");
    $stmt->execute([
        ':startDate' => $startDate,
        ':endDate' => $endDate
    ]);
    return $stmt->fetchAll();
}
```

Latihan 3: Buat Controller Baru

Soal: Buat controller `ReportController` untuk generate laporan PDF.

Petunjuk:

```
class ReportController {
    private Transaction $transactionModel;

    public function __construct(PDO $db) {
        $this->transactionModel = new Transaction($db);
    }

    public function generatePdf() {
        // Implementasi generate PDF
    }
}
```

■ Referensi Tambahan

Dokumentasi PHP OOP

- [PHP Manual - Objects](#)
- [PHP Manual - Classes and Objects](#)

Design Patterns

- [Design Patterns -refactoring.guru](#)
- [SOLID Principles - refactoring.guru](#)

Framework Modern (Setelah OOP Dasar)

- Laravel: <https://laravel.com>
 - Symfony: <https://symfony.com>
 - CodeIgniter 4: <https://codeigniter.com>
-

■ Penutup

Program **PelacakDuit** adalah contoh sederhana penerapan PHP OOP yang cocok untuk mahasiswa yang baru belajar konsep Object-Oriented Programming. Dengan mempelajari struktur kode, konsep, dan praktik di aplikasi ini, mahasiswa dapat memahami:

- Cara kerja class dan object
- Prinsip enkapsulasi dan dependency injection

- Pattern MVC untuk pemisahan logic dan presentation
- Autoloading dan namespace di PHP
- PDO untuk database interaction (anti-SQL injection)
- Prinsip Single Responsibility

Catatan penting:

- Program ini **tidak memiliki fitur login/autentikasi** (untuk pembelajaran saja)
- Jangan gunakan untuk production tanpa menambahkan layer keamanan
- Gunakan program ini sebagai fondasi belajar, lalu kembangkan sesuai kebutuhan

Program ini dibuat dengan AI melalui kerja sama:

- **Hermes Agent** (by Nous Research)
- **9Router** dengan berbagai model AI

Semoga bermanfaat untuk pembelajaran Anda! ■

Dibuat pada: September 2026

Versi: 1.0

Dibuat oleh: Buleun (AI Assistant)

Model: Hermes Agent + 9Router

Dokumen ini dibuat secara otomatis dari markdown menggunakan Python reportlab.
Program PelacakDuit dibuat dengan bantuan AI: Hermes Agent + 9Router.